

Story 16: 查询患者心率的变化趋势

1. 1. Story功能描述

1.1. 系统名称

生命体征监测系统 - 心率趋势监测模块

1.2. 类型

医疗数据分析功能

1.3. 在项目中的地位

作为生命体征监测系统的核心分析功能之一，负责患者心率数据的趋势分析，为医护人员提供实时监测和预警支持。

1.4. 任务目标

- 展示患者心率数据的时间序列变化趋势
- 计算并显示心率的统计数据
- 支持多时间范围的数据查询
- 提供数据可视化分析

2. 2. 详细设计

2.1. 2.1 界面

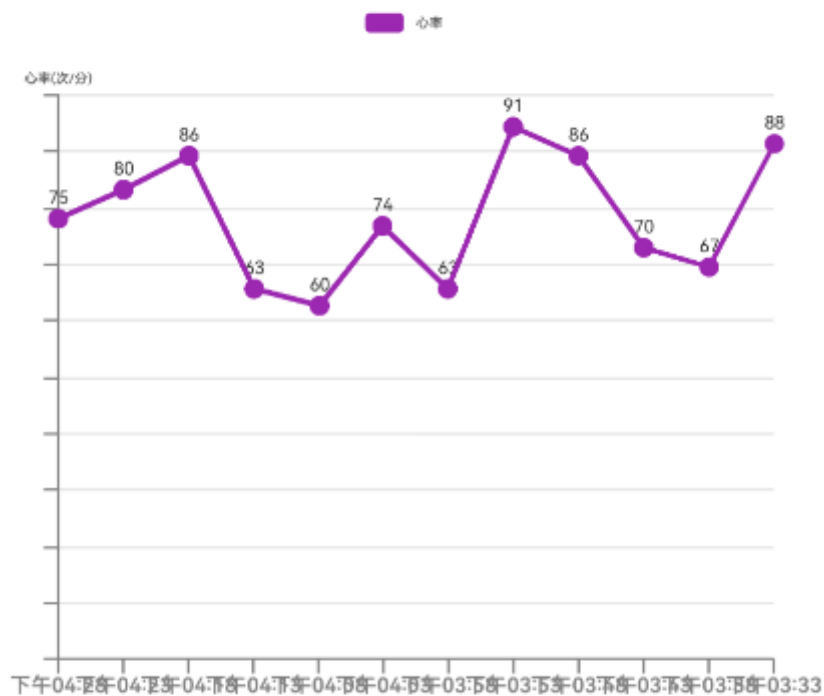
11:02

100

心率趋势

心率趋势

2025/4/18



91 次/分
最高值

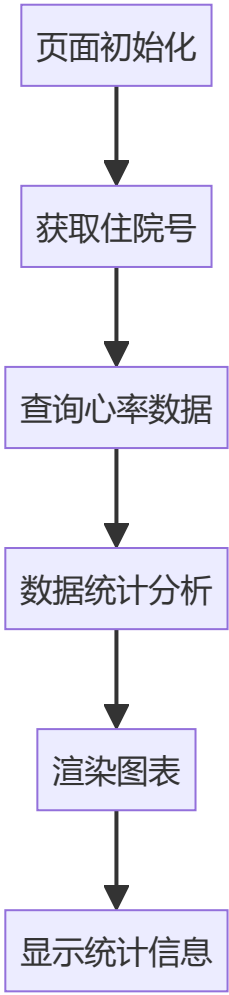
75 次/分
平均值

60 次/分
最低值

查看详细数据

```
1 Column() {
2   // 顶部导航栏
3   Row() {
4     Image($r('app.media.ic_public_arrow_left'))
5     Text('心率趋势')
6   }
7
8   // 图表区域
9   McLineChart({
10    xAxis: { type: 'category', data: timeData },
11    series: [{
12      name: '心率',
13      type: 'line',
14      data: heartRateData,
15      color: '#9C27B0'
16    }]
17  })
18
19  // 统计数据区
20  Row() {
21    StatisticCard('最高值', `${maxHeartRate}次/分`)
22    StatisticCard('平均值', `${avgHeartRate}次/分`)
23    StatisticCard('最低值', `${minHeartRate}次/分`)
24  }
25 }
```

2.2. 2.2 内部处理流程



2.3. 2.3 数据流

输入：

```
1 interface HeartRateData {
2   value: number;           // 心率值
3   measureTime: string;    // 测量时间
4   hospitalId: string;     // 住院号
5 }
```

输出：

```
1 interface HeartRateStats {
2   highest: number;        // 最高心率
3   lowest: number;         // 最低心率
4   average: number;        // 平均心率
5 }
```

2.4. 2.4 算法描述

```
1 function calculateHeartRateStats(data: HeartRateData[]): HeartRateStats {
2   // 初始化统计变量
3   let maxHeartRate = data[0].value;
4   let minHeartRate = data[0].value;
5   let sumHeartRate = 0;
6
7   // 遍历计算
8   for (const record of data) {
9     maxHeartRate = Math.max(maxHeartRate, record.value);
10    minHeartRate = Math.min(minHeartRate, record.value);
11    sumHeartRate += record.value;
12  }
13
14  // 计算平均值
15  const avgHeartRate = Math.round(sumHeartRate / data.length);
16
17  return {
18    highest: maxHeartRate,
19    lowest: minHeartRate,
20    average: avgHeartRate
21  };
22 }
```

2.5. 2.5 外部接口

```
1 interface VitalSignsDB {
2   // 查询指定住院号患者的心率数据
3   getHeartRatesByHospitalId(hospitalId: string): Promise<HeartRateData[]>;
4 }
5
6 interface RouterInterface {
7   // 页面导航接口
8   pushUrl(params: {
9     url: string;
10    params: {
11      hospitalId: string;
12      type: 'heartRate';
13    };
14  }): void;
15 }
```

2.6. 2.6 其他设计考虑

2.6.1. 性能设计

- 使用 McLineChart 组件优化大数据量渲染
- 实现数据分页加载机制
- 缓存已加载的数据减少查询

2.6.2. 安全设计

- 数据访问权限控制
- 敏感信息加密存储
- 操作日志记录

2.6.3. 可靠性设计

- 网络异常重试机制
- 数据加载失败提示
- 异常数据处理
- 心率异常值预警机制